

REMU: Memory-aware Radiation Emulation via Dual Addressing for In-orbit Deep Learning System

Longnv Xu[†], Meiqi Wang[†], Han Qiu*, Jun Liu, Yuanjie Li, Hewu Li*

Tsinghua University, Beijing, China

Email: {xln23@mails., qiuhan@}tsinghua.edu.cn

Abstract—The deployment of commercial-off-the-shelf (COTS) GPUs in space has emerged as a promising approach for supporting in-orbit deep neural network (DNN) inference. However, unlike terrestrial environments, understanding the impact of space radiation on COTS GPU-enabled DNNs is critical. This is challenging because existing methods, such as real-world radiation testing and software emulation, fail to link radiation-induced memory errors to runtime DNN behaviors. In this paper, we propose REMU, a memory-aware Radiation EMULATOR to fill this gap. REMU introduces a dual addressing mechanism across virtual, physical, and DRAM memory spaces, enabling precise mapping and efficient injection of radiation-induced errors from DRAM to runtime DNN inference. Extensive evaluations across 10 well-known DNN models and 2 typical in-orbit computing tasks demonstrate the effectiveness of REMU, providing valuable insights for understanding the resilience of runtime DNN inferences on space radiations.

Index Terms—Deep learning, AI on satellites, memory emulation.

I Introduction

In-orbit computing [1], [2] has emerged as a novel form of “edge computing”, where data is processed directly on satellites to reduce downlink bandwidth [3], [4] or to perform latency-sensitive tasks in orbit, such as wildfire monitoring [5]. In order to support complex data processing tasks like deep neural networks (DNNs), a promising trend [6] is to launch commercial-off-the-shelf (COTS) GPUs into space (see statistics from public information in Fig. 1) instead of relying on radiation-harden chips which are not only 10× more expensive [7] but also unable to support DNN inference [8].

However, different from terrestrial edge computing scenarios, COTS GPUs (e.g., NVIDIA NX) without radiation hardening are vulnerable to radiation-induced errors in space. On the one hand, both ESA’s 10-year in-orbit experimental records [9] and various ground-based radiation tests [10], [11] have demonstrated that space radiation particles can induce bit flips in COTS memory. These include single-event upsets (SEUs), which flip a single bit, and multiple-cell upsets (MCUs), which flip multiple spatial-correlated bits, occurring at a non-negligible rate (e.g., at least 1×10^{-7} /bit/day [12]). On the other hand, runtime DNN models are especially susceptible since their memory-intensive data flow amplifies the probability of error accumulation during critical computations. Current efforts to investigate radiation-induced errors can be classified into two approaches. (1) Real-world validation rely on beam tests [11] or in-orbit flight experiments [12], which requires significant resources and time (e.g., years in [9]), to obtain physical error distributions on memory cells. However, there is still a huge gap between memory hardware errors and the DNN inference behaviors. (2) Software-based error injection offers a cost-effective alternative [13] but existing tools cannot be directly adopted to evaluate how radiation-induced errors influence COTS GPU-based DNN inference. Particularly, they either rely on complex instrumentation close to the hardware [14]–[16] or adopt naïve error injection in static DNN models [17]–[19], regardless of the characteristics of hardware errors and the runtime dynamics of

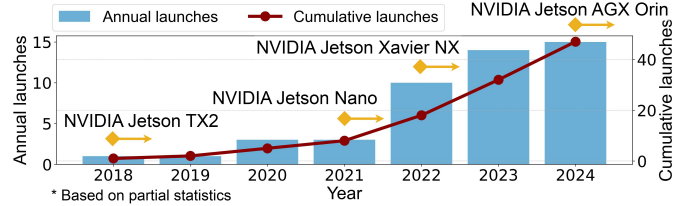


Fig. 1: COTS GPUs launched into space in recent years.

DNN inference. Therefore, we summarize the challenge as follows: *how to rigorously understand the radiation-induced errors’ impact on those COTS GPU-enabled runtime DNN’s performance.*

In this paper, we tackle the above challenge by proposing REMU, a memory-aware Radiation EMULATOR that bridges radiation-induced memory hardware errors to runtime DNN’s behavior. (1) REMU can *precisely* trace memory bit errors across the system (i.e., from low-level DRAM to high-level process space) to realize an end-to-end error mapping. (2) To *efficiently* navigate error tracking in massive DNN memory usage, we develop a novel dual-addressing mechanism using bitmap-based trees, reducing error injection overhead from $\sim 100\times$ to $\sim 3\times$. (3) REMU supports mainstream satellite payload platforms (e.g., TensorRT on NVIDIA Jetson series [20]) but can also be *extended* with other configurations.

Based on REMU, we investigate how COTS GPU-enabled DNN inference’s performance is influenced by radiation-induced memory errors with 10 well-known DNN models on 2 typical in-orbit computing tasks. Our 3 key findings are as follows. (1) Different DNN model structures exhibit different inherent error-tolerance to radiation-induced errors (i.e., lightweight models such as EfficientNet [21] are more sensitive while compacted models such as ViT [22] are robust). (2) Some uncovered sensitivities in runtime DNN inference can lead to a more severe accuracy collapse (i.e., degrading decision-making to a random guess) than the DNN model’s parameter errors. (3) Hardware-layer spatial-correlated errors (i.e., MCU) in memory *cannot* be simplified as multiple single-bit errors (i.e., SEU) since the sensitivity in DNN inference manifests as clustered bits.

II Preliminaries

A. Space radiation is a threat

Harsh space radiation has emerged as a potent “adversary” for in-orbit computing. Satellites in space are surrounded by radiation particles mainly consisting of trapped protons, secondary neutrons from nuclear interactions, and high-energy cosmic rays [13]. Radiation affects in-orbit electronic components, especially the COTS devices that are rarely hardened for lower cost, causing permanent damage (e.g., degradation and burning) or transient damage (e.g., logical state flip) in the unprotected memory¹ [24]. The transient damages,

¹The main memory chips in current in-orbit COTS devices mostly lack support for error-correcting code (ECC) and are not protected by other hardening techniques, such as triple modular redundancy and shielding [23].

[†]Co-first authors.

*Corresponding authors.

although seemingly trivial, are more prevalent and threatening for in-orbit computing since they can stealthily inject wrong control logic and data values into the computing process and disrupt the outcomes, such as the decision-making results of DNN inference. These errors in memory can be classified into two categories.

Single-event upset (SEU) is induced by ionizing radiation striking that makes electron-hole pairs in transistor gates generate and diffuse, causing a state alteration in a memory cell, i.e., a single-bit error. The probability of SEUs can reach 10^{-7} to 10^{-6} /bit/day [9]. Even one SEU may damage the performance of in-orbit DNN inference.

Multiple-cell upset (MCU) occurs when neutron particles react with impurities and produce a large number of secondary particles with sufficient energy [25], leading to multiple memory cells flipped simultaneously as shown in Fig. 2, i.e., a spatial correlated multi-bit error. In space, MCUs happen more frequently in COTS due to the advanced nanotechnology and more compact chips in size [25].

Existing research [26] has shown that the reliability of DNN inference is fragile as bit-flips can result in *stealthy* wrong inference results without crashes, which may compromise safety-critical in-orbit applications, such as disaster monitoring and ship detection [27]. Thus, a practical emulation of SEU/MCU errors in DNN inference is significant to discover its sensitivity at scale.

B. Memory system of COTS GPUs on satellites

To emulate the radiation-induced errors, especially spatial correlated MCUs from the low-level memory cells to the upper process space of DNN inference, precise memory system modeling is essential. We consider a commonly-used payload, NVIDIA Jetson Xavier NX [28] with 16GB 128-bit LPDDR4 (JESD209-4 [29]) as its main memory. The modeling bridges the process space to the dynamic random access memory (DRAM) via hierarchical addressing. First, the memory management unit (MMU) translates virtual addresses in the process space into physical addresses by mapping virtual pages to physical frames using memory paging. Second, the memory controller (MC) translates physical addresses into DRAM and accesses the data.

As shown in Fig. 2, DRAM typically consists of six hierarchies, including die, channel, rank, bank, array, and cell. LPDDR4 [29] defines dual-channel dies with 16DQ (data I/O pins for parallel data transfer) per channel of varying memory densities, such as 16Gb. To enhance data parallelism, multiple dual-channel dies can be employed (e.g., 128-bit LPDDR4 utilizes 8 dual-channel dies). Each DARM channel consists of multiple banks, and each bank contains three-dimensional arrays: row-, column-, and DQ-dimension. DRAM cells are organized into arrays, with each cell using a capacitor to store logical states (charging to represent logic 1 and discharging to represent logic 0) and a transistor to control access to the capacitor during read and write operations. Besides, MCU's adjacency manifests in three dimensions: consecutive rows sharing the same bitline, consecutive columns sharing the same wordline, and consecutive DQ bits in a logical word.

C. Existing emulation methods

The practical emulation of radiation-induced errors requires addressing the complex interaction between hardware errors and application behavior, making it challenging to balance *efficiency* and *extensibility*. Most existing emulations focus on injecting errors closer to the hardware (mainly targeting registers and computing pipelines rather than DRAM), such as at the intermediate representation [14], the low-level assembly [15], or the microarchitecture [16]. Although these methods are extensible to most upper applications, the complexity of use leads to inefficient error injection and limits their adaptability for sensitivity analysis of runtime DNN inference.

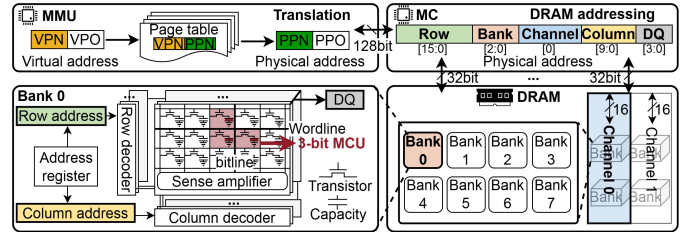


Fig. 2: Hierarchical addressing from process space to DRAM by two steps: translation and DRAM addressing. Detailed organization in a DRAM bank shows MCU's adjacency in three dimensions.

To be more specialized, many error injection tools tailored for deep learning (DL) frameworks have been proposed with expert knowledge, such as TensoFI [17], PyTorchFI [18], and Ares [19]. These methods inject random bit-flip errors directly in DNN models' weights and feature maps for different DL frameworks (e.g., TensorFlow [30] and PyTorch [31]). Although these methods are efficient due to targeted injection in small-scale values, the *offline* model-level error injection lacks critical insights into the runtime process space, such as optimized data flow and actual memory allocation. Thus, it fails to extend to other applications and also misses sensitive points in DNN inference. Besides, all existing methods overlook the impact of spatial correlated MCUs in memory, causing a lack of *correctness*. Thus, we fill the gap to balance the efficiency and extensibility, and also intensify the correctness in our emulation (Sec.III).

III REMU framework

A. Overview

To explore reliable DNN inference under space radiation, emulation of radiation-induced bit-flip errors (i.e., SEUs/MCUs) is imperative. However, existing error injection tools [17]–[19] overlook the spatial correlation of MCUs, leading to impractical error models. To address the gap, we propose REMU (see overview of REMU in Fig. 3) to precisely trace SEU/MCU errors from low-level DRAM to high-level process space. REMU has three main goals to achieve. (1) *Correctness*: SEU/MCU errors occurring in the DRAM must be precisely addressed in the upper DNN inference (Sec.III-B). (2) *Efficiency*: error injection should be fast to facilitate large-scale testing (Sec.III-C). (3) *Extensibility*: the emulation should extend to various DRAMs and radiation-induced error models (Sec. III-D).

B. DNN to DRAM mapping

To precisely inject SEU/MCU errors during runtime DNN inference, the DNN should be first mapped to DRAM. We observe that the process space of DNN inference is vast and radiation-induced SEU/MCU errors are unpredictable. Thus, we simplify the mapping by focusing exclusively on the *inference-critical, time- and memory-intensive* DNN engine (i.e., an optimized runtime DNN model used to execute inference, more details are in Sec.IV-A), which is allocated in the main memory. We realize the mapping via two steps.

First, we leverage the kernel interface (e.g., `/proc/pid/pagemap`²) to translate the userspace virtual addresses to physical addresses (① in Fig. 3). In modern memory management, such as memory paging, allocations are separate due to the fragmented and limited physical memory left when the process is running. Thus, a contiguous virtual memory space of the DNN engine is typically allocated to scattered physical frames (constituting a huge space) with the order varying

²<https://www.kernel.org/doc/Documentation/vm/pagemap.txt> in Linux.

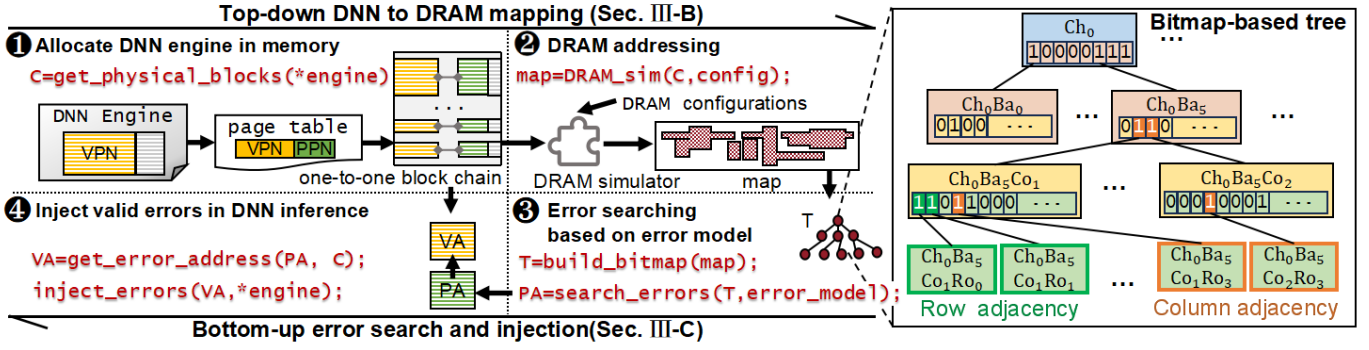


Fig. 3: Overview of REMU for four-step error injection: ❶ Allocate DNN engine’s process space to physical blocks and construct a one-to-one mapping chain of virtual and physical blocks. ❷ Configure the DRAM simulator and perform DRAM addressing based on the addresses in physical blocks to get a vast memory space (i.e., map). ❸ Build the bitmap-based tree based on the physical blocks and search spatial adjacent errors in DRAM via traversal. ❹ Reversely convert valid error physical addresses (PA) to DNN inference’s process space (VA).

from the virtual pages. We construct a one-to-one mapping chain of virtual and physical blocks via the process-context page table.

Second, we leverage the correctness-verified extensible DRAM simulator, Ramulator [32], to address the translated physical memory space in DRAM (❷ in Fig. 3). This benefits from generalized modeling of the DRAM system and extending by configurable DRAM standards (e.g., LPDDR4 [29] in this paper) and DRAM addressing. We improve it to further support the emulation of error injection in DRAM. Specifically, we determine the range of each DRAM level based on the target physical memory space. For example, in a 4KB space, only the lowest 12 bits of the physical address serve for addressing, typically containing 2^0 row and 2^8 columns in LPDDR4. Then, SEUs/MCUs can be emulated in the range and flip the corresponding bit in the virtual memory space. The above two-step extraction from a real memory system can achieve the correct mapping from DNN inference to DRAM.

C. Dual addressing

When executing DNN inference, memory accesses follow a top-down manner from the virtual memory space, translating to the physical memory space, and finally addressing the DRAM cells (Sec.III-B). However, when radiation-induced errors occur in DRAM, their impact on DNN inference flows in an opposite direction. The fragmented physical blocks cause those random errors to get lost in the wilderness (i.e., a vast physical memory space irrelevant to DNN inference). Thus, REMU introduces a dual addressing mechanism utilizing bitmap-based trees to optimize the addressing process and narrow the searching space of relevant errors, which can efficiently inject SEU/MCU errors in DNN inference among the tremendously vast physical memory space (❸ in Fig. 3).

Bitmap-based tree. We establish the bitmap-based tree by traversing all physical address blocks and performing DRAM addressing of these addresses, in which each level of the tree represents a DRAM level and each node in the level contains a bitmap to index the next level. For example, the bitmap 10000111 in Ch_0 indexes $Ba_{0,5,6,7}$, indicating that these banks in the channel are associated with the physical blocks allocated by the DNN inference engine, in which errors are valid. As mentioned in Sec.II-B, MCU’s adjacency manifests in three dimensions: rows, columns, and DQs. For a fast search, we place the *static* bank and channel levels closer to the root and the *dynamic* row, column, and DQ levels closer to the leaves. Thus, the subsequent adjacency checks of low-level nodes (i.e., row and column) will not impact the initial fast selection of high-level nodes (i.e., channel and bank) in a *depth-first traversal*. The search

for a valid error mainly consists of two steps as illustrated in Fig. 3.

First, randomly select a high-level node based on its parent’s bitmap. For example, select Ba_0 based on the bitmap in Ch_0 . Second, select a low-level node based on the error type. 1) For SEU, the selection of a row and column is the same as the high-level node. 2) For column- and row-adjacent MCUs, the selection of columns and rows is based on the adjacency indices (i.e., 11) in their parent’s bitmap. If no adjacency is found in the bitmap, the adjacency check will be continuously performed on the parent’s sibling nodes adhering to the depth-first principle. Once the adjacent columns are determined, the rows at the next level can be selected based on the bitwise *AND* operation of these columns’ bitmaps. This two-step search can efficiently identify a valid SEU/MCU error in DRAM. However, if only one error is searched per traversal, injecting multiple errors remains costly. Thus, we calculate the memory size of the sub-tree for each high-level node and assign the number of errors layer by layer, starting from the root. For example, Ch_0 has 10M address space and Ch_1 has 20M, approximately 1/3 of the SEUs/MCUs falling in Ch_0 and 2/3 in Ch_1 . Therefore, all errors can be searched in a single depth-first traversal.

After error searching in physical memory space, we obtain a series of valid physical addresses that store data relevant to DNN inference, which should be further converted into valid virtual addresses using the previously mentioned one-to-one mapping chain between virtual and physical blocks (❹ in Fig. 3). Additionally, We optimize the mapping by combining several consecutive physical blocks into a large one, thereby further reducing the search complexity.

Complexity analysis. By decomposing the vast linear physical memory space into a nonlinear, bitmap-based tree, the complexity of error searching is significantly reduced. The reasons are two-fold. First, building a tree based on valid physical addresses and searching within it can eliminate the time overhead caused by rechecking invalid errors. Second, spatially adjacent MCUs in DRAM may span distant addresses due to data parallelism and row addressing by high bits of the physical address, making them tricky to map with a simple linear structure. Specifically, the complexity of error searching consists of two parts: error searching in the bitmap-based tree, and error converting from physical address space to virtual address space. The first one reduces from $O(mn)$ to $O(n)$, where n represents the number of valid physical addresses and m denotes the number of attempts on invalid addresses (e.g., for $k = 100$, m typically exceeds 30,000). The second one reduces from $O(n)$ to $O(s)$, where s represents the number of physical blocks (typically thousands in practice) with $O(1)$ addressing within each block using offsets.

D. Extension for usage

REMU requires extensive characterization of the target DRAM device and error model, which is achieved by user-friendly configurations. Following [32], we use external files of the target DRAM device (e.g., *DRAM standard file* and *DRAM addressing file*) as inputs to set up the DRAM simulator. For the default error model evaluated in this paper, we statistically configure SEUs and MCUs based on the extensive in-orbit and ground radiation tests [11]. SEUs are spatially independent and thus can be modeled as random bit-flips. MCUs are spatial adjacent along three dimensions (i.e., row, column, and DQ) with varying multiplicity (e.g., typically ranging from 2-bit to 8-bit), proportions, and patterns. For instance, the probability of flipping two DRAM cells along one wordline is higher (80%) than along one bitline (20%) [11]. The default error model is a combination of SEUs and MCUs with more details in Sec.IV-A. Finally, we pursue REMU in ease of use via source code instrumentation linking with *libREMU_mem.so* library to automate memory addressing and inject SEUs/MCUs in the runtime DNN inference.

IV Experiments and results

A. Setup

DNN models. We describe the DNN baselines used for evaluating the effectiveness of REMU in uncovering radiation-induced sensitivities. Table I lists 10 commonly-used DNN models we evaluate. We target the classical Earth observation dataset, RESISC45 [33], and 7 DNN models for image classification tasks in orbit. We perform an extensive evaluation of diverse DNN backbones, covering traditional CNN-based (e.g., ResNet-50 [34], DenseNet-161 [35]), recent transformer-based structures (e.g., ViT [22]), also the lightweight models widely used on resource-constrained edge (e.g., EfficientNet [21] and MobileNet [36]), and later CNN/MLP variants (e.g., ConvNext [37] and MLP Mixer [38]). Furthermore, we leverage a large-scale dataset, DOTA [39], and 3 variants (i.e., small, medium, large) of a practical DNN detector, YOLOv5 [40], for object detection tasks in orbit.

DNN engines. All evaluated DNN models are converted into TensorRT engines [20] and inferred on real satellite payloads, NVIDIA Jetson Xavier NX [28] with 16GB 128-bit LPDDR4 [29], using TensorRT runtime [20]. A prerequisite step of inference is to convert the trained DNN model from a DL framework (e.g., PyTorch) into optimized data flow graphs and build inference engines (in the main memory³). The TensorRT runtime then creates an execution context from the engine with input and output buffers in preparation for inference. Thus, the engines are crucial for the reliability of DNN inference. Table I shows the size of each engine allocated in memory in preparation for processing one input, which is a good indicator of the memory intensity of each DNN inference. Additionally, the accuracy of all DNN models' inferences without any error is listed in Table I. We further employ low-precision INT8 types to represent quantized floating point values (both parameters and activations) and utilize post-training quantization to create quantized DNN engines. This can achieve faster execution speed ($\approx 4\times$) and lower memory usage ($\approx 1/4$) than full-precision, although incurring a slight accuracy degradation, which is not the priority of our study.

REMU configurations. We account for the bit error rate (BER) and the error model when injecting errors in runtime DNN inference via REMU. We set BER at 10^{-7} and 10^{-6} based on the statistics of existing real-world radiation tests [11], and the number of bit errors depends on the memory intensity of each inference, i.e.,

³In NVIDIA Jetson Xavier NX, the main memory and GPU memory are physically unified. So no typical data copy/transfer between them is needed.

TABLE I: DNN models used in our evaluations. The listed engines are optimized by TensorRT [20] for both FP32 full-precision and INT8 low-precision quantization.

Dataset	Model	Engine size		Baseline accuracy	
		FP32	INT8	FP32	INT8
RESISC45	ResNet-50	115MB	24MB	96.7%	96.6%
	DenseNet-161	144MB	32MB	96.0%	95.0%
	EfficientNet	18MB	6.9MB	91.8%	87.2%
	MobileNet	9.6MB	3.6MB	93.0%	92.2%
	ViT	328MB	107MB	97.1%	95.6%
	MLPMixer	227MB	63MB	95.7%	93.8%
DOTA	ConvNext	108MB	30MB	96.8%	93.1%
	YOLOv5s*	48MB	11MB	81.2%	76.2%
	YOLOv5m*	129MB	28MB	80.4%	75.8%
	YOLOv5l*	294MB	54MB	76.6%	77.3%

*Object detection uses mean average precision (mAP) instead of accuracy.

the size of each engine in this paper. The radiation-induced error model is a configurable input of REMU. In Sec.IV-B, we use the default one (Sec.III-D), to evaluate the error tolerance, which is a mixture of SEUs and MCUs. Fig. 5 shows the detailed proportion of SEUs/MCUs and multiplicity of MCUs for each BER and DNN model. Typically, the proportion of SEUs is higher than 90% and the proportion of ≥ 3 -bit MCUs is significantly less than SEUs and 2-bit MCUs. In Sec.IV-C, we apply six additional error models to illustrate the impacts of MCUs. To better understand REMU's memory-aware error injections, we show a case in Fig. 4. We locate the emulated SEUs and MCUs in both the virtual and physical address space. Specifically, we visualize the lifecycle of two MCU examples during REMU's emulation from the DRAM cells to the inference process, which benefits the precise location of MCUs. Note that we use the default DRAM addressing without interleaved mode [41], thus the bits in an MCU will be kept in one contiguous physical address block. Other address mappings can be customized in REMU like [32].

B. Error tolerance of DNN baselines

To better understand the error tolerance of each DNN model and explore the effectiveness of REMU in identifying sensitivities during each DNN inference, we conduct an extensive evaluation under millions of REMU's error injection. This shows us how differences in BERs, quantization, DNN models, error locations, and error models (Sec.IV-C) can potentially affect the reliability of DNN inference. To our knowledge, this is the first demonstration of DNN inference's error tolerance on runtime memory under near-to-real radiation-induced error emulation, which benefits from REMU's *practical error models* and *precise memory addressing*.

Avoid crashes. Fig. 7 illustrates the impact of SEU and MCU errors on the accuracy of DNN inference. We use REMU to execute 1000 error-injected inferences per BER per DNN model and show the average accuracy. However, we observe a high probability of crashes (e.g., aborts, CUDA failures) during runtime error injections when considering the entire space of the inference engine, which is not our concern. Our concern is a more stealthy error behavior with seamlessly completed inference but wrong results, making errors more difficult to discover. By dumping the engine, we find that errors at the end of the engine⁴ are more likely to activate a crash. Therefore, we additionally omit these areas from the engine and evaluate the error tolerance of DNN inference when errors are located in the remaining space (the majority), denoted as *parameter space*

⁴Engine's serialized format is close-sourced. By dumping the engine, we infer that its end likely contains network definition, engine metadata, executable code of CUDA kernel, etc., associated with inference.

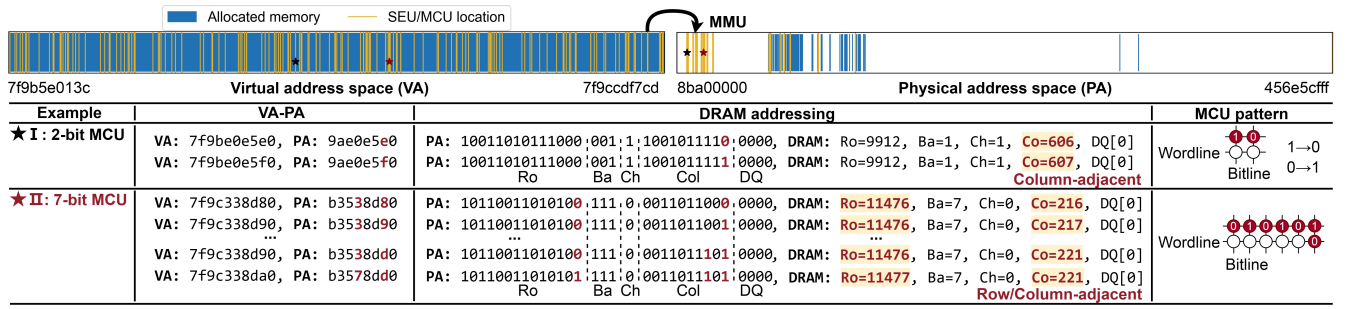


Fig. 4: A case of REMU's error injection. We use REMU to inject SEUs/MCUs in ResNet-50's runtime inference under 10^{-6} BER and the default error model. We show the continuous virtual address space of the inference engine allocated in the main memory and the corresponding discrete physical address space. The seemingly scattered SEUs/MCUs are more clustered in the physical address space. Besides, we give two MCU examples (e.g., 2-bit and 7-bit MCUs) to illustrate the REMU's precise memory addressing from the upper-level process space to the low-level DRAM cells in both row- and column-adjacency.

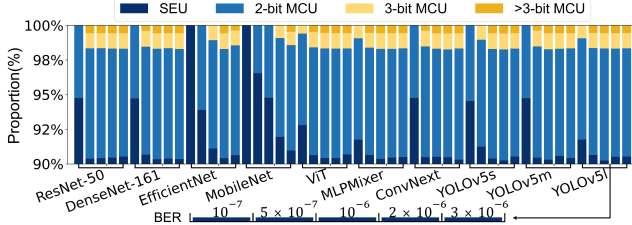


Fig. 5: The proportion of SEUs and the MCUs with different multiplicity in the default error model used for each DNN model and BER. The y-axis is the proportion and the x-axis corresponds to the DNN models (5 BER settings for each).

(★), which results in fewer crashes. Fig. 6 lists the size and crash occurrence of the parameter space compared to the overall engine for each DNN model. Even with this additional evaluation, we underscore that injecting the overall engine is more realistic, as radiation can affect any memory space. When evaluating the overall engine, we skip the crashes and only record the completed inferences.

Quantization. Fig. 7a shows that precision affects the error tolerance of DNN inference, however, the results are not as expected when injecting errors solely in parameter space (★), indicating that the engine additionally holds critical data related to inference beyond the parameter space, which is overlooked in previous model-level error injection tools [17]–[19]. FP32 and FP16 precision exhibit a weaker error tolerance than INT8 precision due to the larger data range and fragile exponent bit. In the following, we focus on INT8 precision as a more challenging case for error attribution.

Accuracy collapse. Fig. 7 shows that the average accuracy declines across almost all DNN models as the BER increases. We observe a significant accuracy drop when injecting errors in the overall engine. For example, the average accuracy drops 18% in DenseNet-161 under 3×10^{-6} BER. If the BER continues increasing, the average accuracy will drop dramatically. Besides, Table II lists the minimum accuracy that occurred during tests, which means DNN inference may experience severe accuracy collapse under radiation-induced errors. Almost all DNN models may be degraded as random guesses (i.e., below 5%) even under a very low BER.

Takeaways. We demonstrate REMU's effectiveness in robust model selection and sensitivity discovery, providing insights for radiation testing and hardening before in-orbit deployment.

First, REMU can support all DNN models and select a better one balancing efficiency, accuracy, and tolerance. In Fig. 7b, REMU reveals that different DNN models exhibit varying sensitivity, e.g., DenseNet-161 (-18%) is more sensitive than ViT (-6%). We observe that the smaller the model, the more sensitive it is. ViT and MLPmixer are relatively more robust but less efficient than the lightweight EfficientNet and MobileNet. Conversely in YOLOv5, the small variant

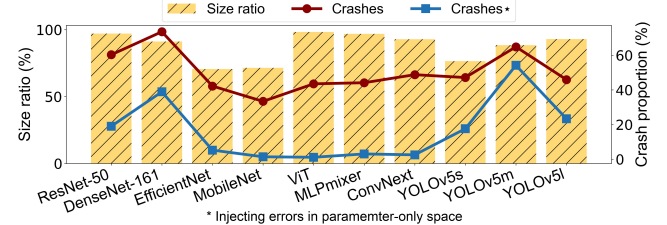


Fig. 6: The left y-axis represents the ratio of the parameter space size to the overall engine size. The right y-axis is the proportion of crashes observed in 1000 inferences. The x-axis corresponds to the DNN models.

is less sensitive but may suffer a more severe accuracy collapse (i.e., 5.4%) than the medium/large. Therefore, when in-orbit edges lean towards smaller models due to resource constraints, we advocate greater efforts on the error tolerance of these models.

Second, REMU can effectively expose the undiscovered sensitivity in the engine. The accuracy collapse in Table II can be located in the engine for further protection. In the parameter space, different models have different locations. For instance, the collapse (27%) occurs in the No.4 layer of EfficientNet but in the No.2 layer of ResNet-50. Beyond the parameter space, the most severe collapse below 5% consistently occurs between two layers in the network definition at the end of the engine as illustrated in Table IIa. These special errors are not intuitively in parameters but auxiliary runtime logic related to the computation of inference. The collapse of 4% highlighted in blue results from the final output buffer not being processed and remaining initialized 0 due to errors. For YOLOv5, such collapse is rare because the post-processing in object detection will further filter out the abnormal output.

C. One MCU does not equal to multiple SEUs

To further investigate the superiority of REMU's memory awareness over previous random error injection [17]–[19], we evaluate the in-depth impact of MCUs that rely on precise addressing of adjacency. If MCUs are not considered (i.e., the proportion is 0), REMU will effectively degrade to random error injection. This is the first to exploit the MCU's impact on DNN inference.

The proportion of MCUs considered in the above default error model is 4%, among which 2% 2-bit MCUs, and 2% 3-bit MCUs. In this evaluation, we set six additional error models with different MCU multiplicities and proportions under the same BER: no MCUs, 50% 2-bit MCUs, 33% 3-bit MCUs, 25% 4-bit MCUs, 20% 5-bit MCUs, and 14% 7-bit MCUs. Fig. 8 shows that the increase in MCU proportion and multiplicity intensifies the reduction of average accuracy. These results are due to aggregated MCUs hitting nearly continuous sensitive bits of DNN inference, especially when the MCU

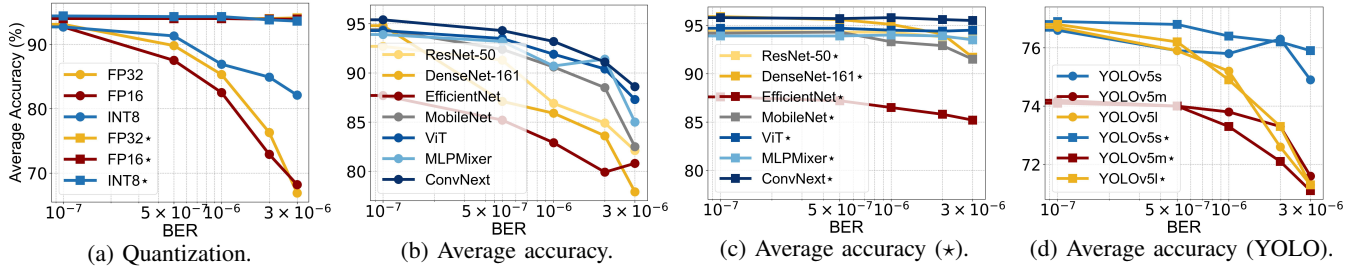


Fig. 7: Average accuracy (y-axis) under REMU's error injections. The x-axis represents BERs. (a) The average accuracy of different quantization levels (e.g., ResNet-50). (b)–(c) The average accuracy of seven classification models when injecting errors in the overall engine and parameter space (*). (d) The average accuracy of three variants of the detection model.

TABLE II: Accuracy collapse under REMU's error injections. (a) A case of collapse (highlighted in blue) in ResNet-50's inference engine. (b)–(c) Accuracy collapse of seven classification models when injecting errors in the overall engine and parameter space (*). (d) Accuracy collapse of three variants of the detection model.

(a) Collapse location.		(b) Accuracy collapse.		(c) Accuracy collapse (*).		(d) Accuracy collapse (YOLO).	
		Model	BER 10 ⁻⁷ 10 ⁻⁶	Model	BER 10 ⁻⁷ 10 ⁻⁶	Model	BER 10 ⁻⁷ 10 ⁻⁶
7f82a37b80(Unnamed La		ResNet-50	1% 1%	ResNet-50*	79% 27%	YOLOv5s	72.1% 43.9%
7f82a37b90 yer* 107) [Scale		DenseNet-161	2% 4%	DenseNet-161*	62% 9%	YOLOv5m	64.8% 50.4%
7f82a37ba0 _output;.....		EfficientNet	4% 1%	EfficientNet*	77% 27%	YOLOv5l	57.0% 57.2%
... ★ 7f82a37c14: fe→7e		MobileNet	1% 1%	MobileNet*	60% 10%	YOLOv5s*	10.0% 5.4%
Accuracy: 4%		ViT	4% 4%	ViT*	93% 41%	YOLOv5m*	63.5% 53.7%
7f82a37c70 ..(Unnamed Layer		MLPMixer	4% 4%	MLPMixer*	29% 10%	YOLOv5l*	51.6% 10.2%
7f82a37c80 * 145) [Activati		ConvNext	4% 4%	ConvNext*	64% 66%		
7f82a37c90 on _output;.....							

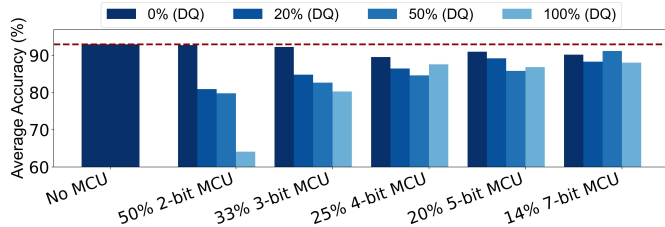


Fig. 8: MCUs' impact on ResNet-50's inference: x-axis is the 6 novel error models with different MCU multiplicities and proportions. For each error model, we set 4 probabilities of MCUs that occur along DQ. The y-axis is the 1000 inferences' average accuracy (10^{-7} BER).

TABLE III: Example of a byte in ResNet-50 inference engine: the accuracy under an SEU error at each bit and the accuracy under a 5-bit MCU error along the DQ-dimension.

Byte (01000001): Accuracy								
0	1	2	3	4	5	6	7	5-bit MCU
94%	42%	42%	42%	42%	42%	94%	94%	9%

diffuses along DQ, in addition to the DRAM rows and columns. We find that bits within a byte (i.e., along DQ) always exhibit identical high sensitivity. For example in Table III, almost every bit in some byte can lead to an accuracy drop, and together resulting in a more severe collapse 9%. To explore these sensitivities, we configure REMU to enable DQ-dimensional MCUs (i.e., set the probability of DQ-dimension greater than 0). Fig. 8 compares MCUs without DQ-dimension and with DQ-dimension of different probabilities for each novel error model. Please note that the proportion of MCU's DQ-dimension diffusion can be modeled via radiation tests. We propose preliminary insights and set the proportion to 0%, 20%, 50%, and 100% in this paper. DNN inference under increased errors along DQ generally shows a worse average accuracy. These results underscore that the introduction of memory addressing in REMU substantially supports the MCU's precise emulation, which is not equal to multiple SEUs and, therefore, can effectively expose more sensitive locations

for further protection, such as aggregated sensitive bits.

D. Other results and discussions

Benefiting from the optimization via the bitmap-based tree, the efficiency of error injection is greatly improved. We test the completion time of DNN inference before/after error injection, also with/without the optimization. The result shows that error injection without optimization introduces a $67.1 - 99.8\times$ completion time increase while the overhead reduces to $3.1 - 3.5\times$ with the optimization. The above findings reveal that runtime DNN inference is highly sensitive to spatially correlated MCU errors and runtime logic. We hope our lessons can foster more academic and industrial efforts on reliable orbital DNN inference, such as allocating critical bits into different memory banks and leveraging detection on output buffers. Our future work will evaluate the sensitivity of DNN inference beyond DRAM and the engine, focusing on instructions and intermediate results.

V Conclusion

This paper designs a memory-aware emulator, REMU, to emulate radiation-induced bit-flip errors, i.e., SEUs and MCUs, and investigate their impact on in-orbit DNN inference. We demonstrate that existing emulation tools either lack easy usage or fail to precisely trace spatially correlated bit errors in memory across hierarchical systems. REMU achieves both correctness and efficiency by precise modeling of the memory system and leveraging the bitmap-based tree for dual addressing to reduce the complexity of large-scale error searching. Based on REMU, extensive experiments are conducted to understand the error behavior of runtime DNN inference, which can benefit the selection of robust model, and sensitivity discovery of the DNN before deployed in orbit.

Acknowledgment

This work is supported by the National Key R&D Program of China (2022YFB3105202), National Natural Science Foundation of China (62272266, 62132009, 62202261), and Center of High performance computing, Tsinghua University.

References

- [1] B. Tao, O. Chabira, I. Janveja, I. Gupta, and D. Vasisht, "Known knowns and unknowns: Near-realtime earth observation via query bifurcation in serval," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 2024, pp. 809–824.
- [2] R. Xing, M. Xu, A. Zhou, Q. Li, Y. Zhang, F. Qian, and S. Wang, "Deciphering the Enigma of Satellite Computing with COTS Devices: Measurement and Analysis," in *ACM International Conference on Mobile Computing and Networking (MobiCom)*, 2024.
- [3] B. Denby, K. Chintalapudi, R. Chandra, B. Lucia, and S. Noghabi, "Kodan: Addressing the computational bottleneck in space," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023.
- [4] B. Denby and B. Lucia, "Orbital edge computing: Machine inference in space," *IEEE Computer Architecture Letters*, pp. 59–62, 2019.
- [5] D. Saaristo, "Shoot from the stars: Startup provides early detection of wildfires from space," <https://blogs.nvidia.com/blog/ororatech-wildfires-from-space/>.
- [6] G. Choumos, K. Tsapralis, V. Lappas, and C. Kontoes, "Artificial intelligence for a safe space: Data and model development trends in orbit prediction and collision avoidance," in *AIAA SCITECH 2024 Forum*, 2024, p. 2066.
- [7] B. Denby and B. Lucia, "Orbital edge computing: Nanosatellite constellations as a new class of computer system," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 939–954.
- [8] M. Esposito, B. Dominguez, M. Pastena, N. Vercruyssen, S. Conticello, C. van Dijk, P. Manzillo, and R. Koeleman, "Highly integration of hyperspectral, thermal and artificial intelligence for the ESA PHISAT-1 mission," in *Proceedings of the International Astronautical Congress IAC, Washington, DC, USA*, 2019, pp. 21–25.
- [9] A. Samaras, F. Bezerra, E. Lorfevre, and R. Ecoffet, "CARMEN-2: In flight observation of non destructive single event phenomena on memories," in *2011 12th European Conference on Radiation and Its Effects on Components and Systems*. IEEE, 2011, pp. 839–848.
- [10] J. P. Walters, K. M. Zick, and M. French, "A practical characterization of a NASA SpaceCube application through fault emulation and laser testing," in *2013 43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2013, pp. 1–8.
- [11] J. C. Fabero, H. Mecha, F. J. Franco, J. A. Clemente, G. Korkian, S. Rey, B. Cheymol, M. Baylac, G. Hubert, and R. Velazco, "Single event upsets under 14-mev neutrons in a 28-nm SRAM-based FPGA in static mode," *IEEE Transactions on Nuclear Science*, pp. 1461–1469, 2020.
- [12] F. Bezerra, R. Ecoffet, E. Lorfevre, A. Samaras, and C. Deneau, "CARMEN2/MEX: An in-flight laboratory for the observation of radiation effects on electronic devices," in *2011 12th European Conference on Radiation and Its Effects on Components and Systems*. IEEE, 2011, pp. 607–614.
- [13] H. Wang, S. Myint, V. Verma, Y. Winetraub, J. Yang, and A. Cidon, "Mars attacks! software protection against space radiation," in *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, 2023.
- [14] Q. Lu, M. Farahani, J. Wei, A. Thomas, and K. Pattabiraman, "Lifi: An intermediate code-level fault injection tool for hardware faults," in *2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 2015, pp. 11–16.
- [15] S. K. S. Hari, T. Tsai, M. Stephenson, S. W. Keckler, and J. Emer, "Sassifi: An architecture-level fault injection tool for gpu application resilience evaluation," in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2017, pp. 249–258.
- [16] O. Chatzopoulos, G. Papadimitriou, V. Karakostas, and D. Gizopoulos, "Gem5-marvel: Microarchitecture-level resilience analysis of heterogeneous soc architectures," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 543–559.
- [17] Z. Chen, N. Narayanan, B. Fang, G. Li, K. Pattabiraman, and N. DeBardeleben, "Tensorfi: A flexible fault injection framework for tensorflow applications," in *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2020, pp. 426–435.
- [18] A. Mahmoud, N. Aggarwal, A. Nobbe, J. R. S. Vicarte, S. V. Adve, C. W. Fletcher, I. Frosio, and S. K. S. Hari, "Pytorchfi: A runtime perturbation tool for dnnns," in *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*. IEEE, 2020, pp. 25–31.
- [19] B. Reagen, U. Gupta, L. Pentecost, P. Whatmough, S. K. Lee, N. Mulholland, D. Brooks, and G.-Y. Wei, "Ares: A framework for quantifying the resilience of deep neural networks," in *Proceedings of the 55th Annual Design Automation Conference*, 2018, pp. 1–6.
- [20] H. Vanholder, "Efficient inference with tensorrt," in *GPU Technology Conference*, vol. 1, no. 2, 2016, pp. 1–24.
- [21] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 6105–6114.
- [22] D. Alexey, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv: 2010.11929*, 2020.
- [23] H. Su, T. Lu, C. Feng, and L. Chen, "Triple module redundancy reliability framework design based on heterogeneous multi-core processor," *Procedia Computer Science*, vol. 183, pp. 504–511, 2021.
- [24] P. Caron, C. Inguibert, L. Artola, R. Ecoffet, and F. Bezerra, "Physical mechanisms of proton-induced single-event upset in integrated memory devices," *IEEE Transactions on Nuclear Science*, vol. 66, no. 7, pp. 1404–1409, 2019.
- [25] A. Dutta and N. A. Touba, "Multiple bit upset tolerant memory using a selective cycle avoidance based sec-ded-daec code," in *25th IEEE VLSI Test Symposium (VTS'07)*. IEEE, 2007, pp. 349–354.
- [26] D. Ma, F. Lin, A. Desmaison, J. Coburn, D. Moore, S. Sankar, and X. Jiao, "Dr. dna: Combating silent data corruptions in deep learning using distribution of neuron activations," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 239–252.
- [27] N. Bleier, M. H. Mubarik, G. R. Swenson, and R. Kumar, "Space micro-datatcenters," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 900–915.
- [28] NVIDIA, "Jetson Xavier NX Developer Kit," 2024, online; accessed 14-March-2024. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-xavier-nx-devkit>
- [29] J. S. S. T. Association *et al.*, "JEDEC standard: Low power double data rate 4 (LPDDR4)," *JEDEC Standard JESD209-4*, 2014.
- [30] M. Abadi, "Tensorflow: learning functions at scale," in *Proceedings of the 21st ACM SIGPLAN international conference on functional programming*, 2016, pp. 1–1.
- [31] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
- [32] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible DRAM simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [33] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proceedings of the IEEE*, vol. 105, no. 10, pp. 1865–1883, 2017.
- [34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [35] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.
- [36] A. G. Howard, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [37] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 11 976–11 986.
- [38] I. O. Tolstikhin, N. Houlsby, A. Kolesnikov, L. Beyer, X. Zhai, T. Unterthiner, J. Yung, A. Steiner, D. Keysers, J. Uszkoreit *et al.*, "Mlp-mixer: An all-mlp architecture for vision," *Advances in neural information processing systems*, vol. 34, pp. 24 261–24 272, 2021.
- [39] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, "Dota: A large-scale dataset for object detection in aerial images," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 3974–3983.
- [40] G. Jocher, "Yolov5 by ultralytics," 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [41] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, "{DRAMA}: Exploiting {DRAM} addressing for {Cross-CPU} attacks," in *25th USENIX security symposium (USENIX security 16)*, 2016, pp. 565–581.